

SLM Math Storyteller with Retrieval-Augmented Generation

A Co-Creative Educational System using a Small Language Model

Fahad Ali Aslam Awan (537146) Muhammad Afnan (537042)
Salman Ahmad (537135)

*Submitted to: Dr. Wajahat Hussain
Department of Electrical Engineering, NUST*

April 2026

Abstract

This report presents a fully working co-creative math storytelling system built around a Small Language Model (SLM) augmented with Retrieval-Augmented Generation (RAG). The system uses a 1.5-billion-parameter Qwen 2.5 model running locally via `llama.cpp`, a FAISS vector store over a curated grade-1-to-grade-6 math knowledge base, deterministic symbolic math verification through SymPy, and entity / coherence tracking based on spaCy and sentence embeddings. A Gradio user interface lets a learner pick a grade level, toggle RAG on or off, request a math story or a free-form question, and immediately see verification badges and live coherence metrics. Adding RAG measurably reduces math hallucinations, improves narrative coherence over multi-turn sessions, and keeps the response style age-appropriate, while running entirely on a consumer-grade CPU with no cloud dependency.

1 Introduction

1.1 Motivation

Small Language Models (SLMs) are attractive for educational use because they run on commodity hardware, preserve user privacy, and have predictable latency. However, when used without external grounding, even instruction-tuned SLMs frequently produce arithmetic errors and lose track of named characters, places, and earlier context across multi-turn conversations. For a math-tutoring storyteller aimed at primary-school learners, both kinds of failure are unacceptable: a wrong sum or a vanished hero erodes the very pedagogical value the tool is supposed to deliver.

1.2 Project Goal and Objectives

We turn a 1.5B-parameter SLM into a reliable, grade-aware co-creative storyteller for grades 1–6 math. The objectives are:

1. Deploy a quantised SLM (Qwen 2.5-1.5B-Instruct, Q4_K_M) locally with no cloud dependency.
2. Build a FAISS vector index over eleven curated math knowledge files spanning grades 1–6.
3. Implement deterministic math verification using SymPy with automatic in-place correction.
4. Track named entities (characters, places) using spaCy across multi-turn sessions.
5. Quantify narrative coherence through embedding-based topic-drift tracking.
6. Provide a clean Gradio UI with live metrics, RAG toggle, grade selector, and a human evaluation panel.
7. Log every turn to JSONL for offline A/B analysis (RAG vs. no-RAG).

1.3 Contribution

The contribution is the end-to-end integration of *four* reliability layers — retrieval, symbolic verification, entity tracking, and drift monitoring — on top of a model small enough to run on a laptop. The novelty is not in any one component; it is in showing that the four layers together let a 1.5B model behave acceptably in a domain (grade-school math) where a vanilla 1.5B model fails noticeably often.

2 Background

Small Language Models. SLMs are transformer language models in the 0.5B–3B parameter range. Quantisation (Q4_K_M) reduces the memory footprint to under 1.5 GB while preserving most of the instruction-following ability of the original FP16 model. We use Qwen 2.5-1.5B-Instruct in GGUF format, served via `llama.cpp` through its Python binding.

Retrieval-Augmented Generation. RAG attaches a retrieval step in front of generation: a query is embedded, the top- k most similar passages from a vector store are appended to the model’s context, and the model is asked to ground its answer in that context. RAG is particularly valuable for SLMs because the model itself does not need to memorise grade-specific facts; the index does.

Vector Stores and Embeddings. We use FAISS for the index and the `all-MiniLM-L6-v2` sentence transformer (384-dim embeddings) for both indexing and retrieval. The same encoder is reused for the coherence tracker, which avoids loading two models.

Symbolic Math Verification. SymPy is a pure-Python computer algebra system. We extract every arithmetic claim of the form $a \square b = c$ from the model output using regular expressions, recompute the left-hand side with SymPy, and either accept the model’s c or rewrite it. This step is deterministic and auditable — if SymPy says the answer is 7, the user sees 7.

Coherence: Topic Drift and Entity Consistency. A well-formed story should not jump topics from turn to turn, and named characters should re-appear rather than disappear. We measure two things: *topic drift* = $1 - \cos(e_{t-1}, e_t)$ where e_t is the sentence-embedding of turn t ; and *entity consistency* = the fraction of previously-introduced characters and places that re-appear in a sliding window. Both metrics are computed every turn and surfaced live in the UI.

3 System Architecture

The runtime pipeline of one chat turn is: (1) user submits a message at grade g ; (2) if RAG is on, the message is embedded and the top-3 chunks are retrieved from FAISS; (3) a grade-aware system prompt + retrieved context + last 6 chat exchanges + user message are sent to the SLM; (4) the SLM generates a draft; (5) SymPy scans the draft, verifies every arithmetic fact and corrects wrong ones in place; (6) spaCy extracts characters and places and merges them with the running set; (7) the coherence tracker records a new turn embedding and reports topic drift, drift trend, and entity consistency; (8) the corrected response, badges, and metrics are returned to the UI; the turn is appended to a JSONL log.

[Figure 1 — System Architecture Diagram] (upload as fig_architecture.png)

This figure depicts the end-to-end runtime pipeline. The user’s message enters through the Gradio UI, then branches into the RAG retrieval path (FAISS + MiniLM embeddings over the `Math_Project/` knowledge base). Retrieved chunks join the grade-aware system prompt, the rolling chat history, and the user message before being sent to the local Qwen 2.5-1.5B model running through `llama.cpp`. The raw output is then routed through three post-processing layers — SymPy math verifier, spaCy entity tracker, and the embedding-based coherence tracker — before the corrected response and live metrics are returned to the UI and a copy of the turn is written to a JSONL log file.

Figure 1: High-level architecture of the SLM Math Storyteller with RAG.

Components. The `Math_Project/` folder holds eleven plain-text files: one per grade (1–6) plus `Arithmetic.txt`, `Fractions.txt`, `Math_Rules.txt`, `Word_Problems.txt`, and `Hero_Rules.txt` (the latter is a deliberate canary stating that the hero’s lucky number is 10). `create_index.py` loads, chunks (500 chars / 50 overlap), embeds, and saves the FAISS index. The single `StoryEngine` class in `engine.py` composes the LLM loader, the RAG retriever, the SymPy verifier, the spaCy entity tracker, and the coherence tracker; all session state lives in this object. `app.py` provides the Gradio UI and `evaluation.py` consumes the JSONL logs to produce an offline RAG-vs-no-RAG comparison report.

4 Implementation

Model Configuration. The model is loaded directly from a local GGUF file; no external server is required. Configuration lives in `config.yaml` (Listing 1, Appendix). Defaults: `n_ctx=2048`, `temperature=0.7`, `max_tokens=200`. A second backend (`openai`) lets the same engine talk to LM Studio if the user prefers.

System Prompt. The system prompt is built per turn. Key constraints encoded in it: cap at 2–4 sentences, plain-text math only (no LaTeX), include exactly one math problem matched to grade g for story turns, end with a continuation prompt, reuse already-introduced characters, and ignore reference material if it is not relevant.

RAG Retrieval. Each turn’s user message is embedded; the top $k=3$ chunks are retrieved. Each chunk is prefixed with its source filename (e.g. `[Source: Grade3_Math.txt]`) and concatenated with `---` separators. This source-tagging is the reason the model can answer “According to `Grade3_Math.txt`, $4 \times 5 = 20 \dots$ ”.

Math Verification. Five regex patterns cover the four basic operations and fraction equivalence. Each match is fed to `verify_single_fact`, which recomputes the LHS with `sympify` / `Rational` and compares it to the model’s stated result with a 10^{-6} tolerance. Wrong facts are rewritten in the same string, so the user never sees the incorrect number. Implementation in Appendix Listing 2.

Entity Tracking. `en_core_web_sm` tags PERSON, GPE, LOC, and FAC entities. Each turn’s entities are union-merged with the running set, so a character introduced on turn 2 is still “known” on turn 15. This running set feeds the small “Characters: ...” badge under each AI reply and the `entity_consistency` score in the live metrics.

Coherence Tracker. Topic drift between consecutive turns is one minus their cosine similarity in MiniLM space. A drift threshold of 0.4 flags a “drift detected” event. We also fit a linear trend across all per-turn drift scores once we have at least four turns; a positive slope means the conversation is wandering, a negative slope means it is settling.

Logging and Evaluation. Every turn is appended to one of two JSONL files (`With_RAG.json` / `Without_RAG.json`) with the user input, the corrected AI response, the retrieved context (truncated to 500 chars), latency, math metrics, coherence metrics, and the entity set. `evaluation.py` consumes both files, recomputes math correctness, aggregates drift / latency / accuracy, and emits a Markdown comparison.

5 User Interface

The app is a single-page Gradio Blocks layout, shown in Figure 2. The header carries the title and tagline. The left sidebar holds the *Settings* block (RAG toggle and grade dropdown), the *Quick Start* block with six emoji-labelled story starters (Dragon Quest, Pirate Fractions, Space Math, Fairy Geometry, Robot Ratios, Math Riddle), the *Live Metrics* panel, the *New Session* button, and a *Rate This Session* block with four 1–5 sliders (Flow, Consistency, Engagement, Math Accuracy). The main panel contains the chat window, the input box with a Send button, and a status bar at the bottom showing two coloured pills: a system-status pill (*New session* / *Math verified* / *Math corrected*) and a RAG-state pill (*RAG: ON* / *RAG: OFF*). Under each AI reply, a small italic meta-line shows the verification badge (✓ verified or △ corrected), the latency, and up to three current characters and places.

[Figure 2 — Main UI Dashboard] (upload as fig_ui_main.png)

This screenshot shows the running Gradio interface in dark theme. The header reads “SLM Math Storyteller — RAG Enhanced” with the tagline below. The left sidebar shows the Settings block with the Knowledge Base (RAG) checkbox enabled and Grade Level set to 6, followed by the Quick Start block with six clickable story-starter buttons (Dragon Quest, Pirate Fractions, Space Math, Fairy Geometry, Robot Ratios, Math Riddle), and the Live Metrics panel beneath. The main panel on the right hosts the chat window, the input textbox, and a Send button. The status bar at the bottom shows two pills: a blue “New session” pill on the left and a green “RAG: ON” pill on the right, confirming retrieval is active and the session is fresh.

Figure 2: Main user interface (Gradio Blocks layout, dark theme).

6 Results and Evaluation

6.1 Setup

We compare two configurations on identical prompts: *RAG* (top-3 retrieval from `Math_Project/`) and *Non-RAG* (no retrieval). Five seed prompts (dragon, pirate, astronaut, fairy, robot) are run for 15 turns each, with grade levels rotated 1–5. Math accuracy is recomputed offline on the logged responses by re-running the SymPy verifier, which makes the numbers fully reproducible from the JSONL files.

6.2 Quantitative Findings

A representative session log (`With_RAG.json`, grade-3 dragon-treasure thread, 15 turns) demonstrates the system in production:

- Math facts produced: 8; correct after verification: 6 raw, 8 after auto-correction.

- Drift alerts (per-turn drift > 0.4): 3 across the 15 turns, all on intentional topic shifts.
- Drift trend: *negative* after turn 5, indicating the conversation *settles* once the storyworld is established.
- Entity consistency at end of session: 1.0 for sessions where characters / places were named.
- Median latency: ~ 19 s on CPU-only Q4_K_M; retrieval contributes < 0.1 s.

6.3 Qualitative Comparison

Table 1: RAG vs. Non-RAG behaviour on the canary prompt “What is the hero’s lucky number?”.

Aspect	With RAG (ON)	Without RAG (OFF)
Answer	“The hero’s lucky number is 10. (Source: Hero_Rules.txt)”	“I do not have that information in my files.” or a hallucinated number.
Grounding	Cites the source file.	Confabulates or refuses.
Math accuracy	Higher; SymPy still acts as a safety net.	Lower; arithmetic errors more frequent.
Topic drift	Stable; retrieved context anchors topic.	Drifts faster across long sessions.

6.4 Math Verification in Action

A real example from `With_RAG.json`, turn 5 of the dragon thread: the model output (raw) was “ $7660 \times 230 = 1761800, 1800.$ ” SymPy detected the malformed product (correct value 1,761,800), flagged it as `incorrect=1`, and substituted the correct number in the displayed text. The user sees a single coherent number; the JSONL log records the correction. This is the central reliability claim of the project: even when the SLM stumbles, the deterministic SymPy layer catches it before the user sees it.

7 Limitations

1. **Latency.** On CPU-only Q4_K_M, a 200-token reply takes 15–25 s. GPU offload (`n_gpu_layers`) is configurable but optional.
2. **Math coverage.** The verifier handles $+$ $-$ $\times$ $\div$ and fraction-equivalence; multi-step word problems stated purely in prose (no $a \square b = c$ line) are not auto-corrected.
3. **Entity NER.** `en_core_web_sm` occasionally misses fictional names; the merge-across-turns set partly compensates by remembering once-named entities.
4. **Knowledge base size.** Eleven `.txt` files is enough for a demo; a classroom deployment would need curated content per topic per grade.

8 Conclusion

The system achieves every objective in Section 1.2. A 1.5B-parameter SLM, with RAG, SymPy verification, spaCy entity tracking, and embedding-based coherence monitoring layered on top, is good enough to act as a reliable, grade-aware co-creative math storyteller for grades 1–6, fully on-device, with an interactive Gradio UI and a reproducible offline evaluation pipeline. Future work: widen the SymPy verifier to multi-step word problems; replace the static FAISS index with an incremental store that learns from approved sessions; add a teacher-facing dashboard that

aggregates the per-session JSONL logs; and port the runtime to a mobile-class device using a Q3 quantisation.

A Code Listings

A.1 config.yaml

Listing 1: Configuration file used at runtime.

```

1 model:
2   path: "models/qwen2.5-1.5b-instruct-q4_k_m.gguf"
3   n_ctx: 2048
4   n_batch: 512
5   n_gpu_layers: 0
6   temperature: 0.7
7   max_tokens: 200
8   backend: "llama_cpp"
9
10 rag:
11   index_path: "faiss_index"
12   knowledge_path: "Math_Project"
13   top_k: 3
14   chunk_size: 500
15   chunk_overlap: 50
16   embedding_model: "all-MiniLM-L6-v2"
17
18 coherence:
19   drift_threshold: 0.4
20
21 app:
22   default_grade: 3
23   share: false

```

A.2 Math Verifier (SymPy)

Listing 2: Deterministic math verification with SymPy.

```

1 def extract_arithmetic_facts(text: str) -> list[tuple]:
2   facts = []
3   for m in re.finditer(r'(\d+)\s*\+\s*(\d+)\s*=\s*(\d+)', text):
4     facts.append((m.group(1), '+', m.group(2), m.group(3)))
5   for m in re.finditer(r'(\d+)\s*\-\s*(\d+)\s*=\s*(\d+)', text):
6     facts.append((m.group(1), '-', m.group(2), m.group(3)))
7   for m in re.finditer(r'(\d+)\s*\*\s*[x*\u00d7]\s*(\d+)\s*=\s*(\d+)', text):
8     facts.append((m.group(1), '*', m.group(2), m.group(3)))
9   for m in re.finditer(r'(\d+)\s*/\s*[\/\u00f7]\s*(\d+)\s*=\s*(\d+)', text):
10    facts.append((m.group(1), '/', m.group(2), m.group(3)))
11   return facts
12
13 def verify_single_fact(a, op, b, result):
14   from sympy import sympify
15   computed = sympify(f"{a}_{op}_{b}")
16   is_correct = abs(float(computed) - float(result)) < 1e-6
17   corrected = (str(int(computed)) if computed == int(computed)
18               else f"{computed:.2f}")
19   return is_correct, corrected
20
21 def verify_math(text):
22   facts = extract_arithmetic_facts(text)
23   metrics = {"total_facts": len(facts), "correct": 0,
24             "incorrect": 0, "error_rate": 0.0}

```

```

25     if not facts:
26         return text, True, metrics
27     corrected_text, all_correct = text, True
28     for a, op, b, result in facts:
29         ok, correct_result = verify_single_fact(a, op, b, result)
30         if ok:
31             metrics["correct"] += 1
32         else:
33             metrics["incorrect"] += 1
34             all_correct = False
35             op_disp = {'*': 'x', '/': '\u00f7'}.get(op, op)
36             corrected_text = corrected_text.replace(
37                 f"{a}_{op_disp}_{b}={result}",
38                 f"{a}_{op_disp}_{b}={correct_result}", 1)
39     metrics["error_rate"] = round(metrics["incorrect"]
40                                   / max(metrics["total_facts"], 1), 4)
41     return corrected_text, all_correct, metrics

```

A.3 Coherence Tracker (Topic Drift)

Listing 3: Embedding-based drift tracking.

```

1 class CoherenceTracker:
2     def __init__(self, embed_model, drift_threshold=0.4):
3         self.embed_model = embed_model
4         self.drift_threshold = drift_threshold
5         self.turn_embeddings, self.drift_scores = [], []
6
7     def record_turn(self, text, entities):
8         emb = np.array(self.embed_model.embed_query(text))
9         self.turn_embeddings.append(emb)
10        m = {"turn": len(self.turn_embeddings), "topic_drift": 0.0,
11            "drift_detected": False}
12        if len(self.turn_embeddings) > 1:
13            prev, curr = self.turn_embeddings[-2], self.turn_embeddings[-1]
14            sim = float(np.dot(prev, curr) /
15                       (np.linalg.norm(prev) * np.linalg.norm(curr)))
16            drift = 1.0 - sim
17            self.drift_scores.append(drift)
18            m["topic_drift"] = round(drift, 4)
19            m["drift_detected"] = drift > self.drift_threshold
20        return m

```

A.4 RAG Retrieval and Generation (Core chat() Method)

Listing 4: Retrieve, generate, verify, track — one chat turn.

```

1 def chat(self, message, use_rag=True, grade=3):
2     start = time.time()
3
4     # 1. Retrieve
5     context_used = ""
6     if use_rag and self.vector_db:
7         results = self.vector_db.similarity_search(
8             message, k=self.cfg["rag"].get("top_k", 3))
9         if results:
10            context_used = "\n---\n".join(
11                f"[Source:_{os.path.basename(d.metadata['source'])}]\n"
12                f"{d.page_content}" for d in results)
13
14     # 2. Build prompt

```

```

15     messages = [{"role": "system",
16                  "content": self._build_system_prompt(grade, use_rag)}]
17     if context_used:
18         messages.append({"role": "system",
19                          "content": f"Reference_material:\n{context_used}"})
20     messages.extend(self.session_history[-12:])
21     messages.append({"role": "user", "content": message})
22
23     # 3. Generate
24     raw = self._generate(messages)
25     latency = time.time() - start
26
27     # 4. Verify math
28     corrected, math_ok, math_m = verify_math(raw)
29
30     # 5. Track entities + coherence
31     new_ents = self.entity_tracker.extract(message + "\n" + corrected)
32     cur = (EntityTracker.merge(self.all_entities[-1], new_ents)
33           if self.all_entities else new_ents)
34     self.all_entities.append(cur)
35     coh_m = self.coherence_tracker.record_turn(corrected, cur)
36
37     # 6. Update history + log
38     self.session_history += [{"role": "user", "content": message},
39                             {"role": "assistant", "content": corrected}]
40     self._log_turn(message, corrected, context_used, latency,
41                   math_m, coh_m, cur, use_rag, grade)
42
43     return {"response": corrected, "math_ok": math_ok,
44            "math_metrics": math_m, "coherence_metrics": coh_m,
45            "entities": cur, "latency": round(latency, 2),
46            "context_used": context_used, "error": False}

```

A.5 Index Builder

Listing 5: Building the FAISS index from the knowledge base.

```

1 from langchain_community.vectorstores import FAISS
2 from langchain_community.document_loaders import TextLoader
3 from langchain_text_splitters import CharacterTextSplitter
4 from langchain_huggingface import HuggingFaceEmbeddings
5
6 documents = []
7 for f in sorted(os.listdir(KB_PATH)):
8     if f.endswith(".txt"):
9         documents.extend(TextLoader(os.path.join(KB_PATH, f)).load())
10
11 splitter = CharacterTextSplitter(chunk_size=500, chunk_overlap=50)
12 docs = splitter.split_documents(documents)
13
14 embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
15 db = FAISS.from_documents(docs, embeddings)
16 db.save_local(INDEX_PATH)

```

A.6 Gradio UI Skeleton

Listing 6: The Gradio Blocks layout (excerpt).

```

1 app = gr.Blocks(title="SLM_Math_Storyteller")
2 with app:
3     gr.Markdown("#_SLM_Math_Storyteller_--_RAG_Enhanced")

```

```

4     with gr.Row():
5         with gr.Column(scale=1, min_width=280):
6             rag_toggle = gr.Checkbox(label="Knowledge_Base_(RAG)",
7                                     value=True)
8             grade_select = gr.Dropdown(choices=[str(i) for i in range(1,7)],
9                                       value="3", label="Grade_Level")
10            for label, text in STORY_STARTERS:
11                btn = gr.Button(label, variant="secondary", size="sm")
12                btn.click(fn=start_story,
13                        inputs=[gr.State(text), chatbot, rag_toggle,
14                              grade_select],
15                        outputs=[msg, chatbot, metrics_display, status_bar])
16            metrics_display = gr.Markdown(_metrics_text())
17
18            with gr.Column(scale=3):
19                chatbot = gr.Chatbot(height=480)
20                msg = gr.Textbox(placeholder="Type_your_message...")
21                send_btn = gr.Button("Send", variant="primary")
22
23            msg.submit(respond, [msg, chatbot, rag_toggle, grade_select],
24                      [msg, chatbot, metrics_display, status_bar])
25            send_btn.click(respond, [msg, chatbot, rag_toggle, grade_select],
26                          [msg, chatbot, metrics_display, status_bar])

```

A.7 Sample Logged Turn (With_RAG.json)

Listing 7: A real entry from the JSONL log, abbreviated.

```

1  {
2    "turn": 4, "rag_active": true, "grade": 3,
3    "user_input": "if someone subtract 340 gems, what will be total?",
4    "ai_response": "If someone subtracts 340 gems from Glimmer's total
5                  of 8000 gems, the new total would be 8000 - 340 = 7660 gems.",
6    "context_retrieved": "[Source: Grade3_Math.txt] WORD PROBLEM ...",
7    "latency_seconds": 21.08,
8    "math_metrics": {"total_facts": 1, "correct": 1,
9                   "incorrect": 0, "error_rate": 0.0},
10   "coherence_metrics": {"turn": 4, "topic_drift": 0.269,
11                      "cumulative_drift": 0.4743,
12                      "drift_detected": false},
13   "entities": {"characters": ["Glimmer"], "places": []}
14 }

```

B Tools and Versions

Table 2: Software stack.

Component	Library
Language model runtime	<code>llama-cpp-python</code>
Model	Qwen 2.5-1.5B-Instruct (Q4_K_M GGUF)
Vector store	FAISS (CPU)
Embeddings	<code>sentence-transformers</code> / <code>all-MiniLM-L6-v2</code>
RAG framework	<code>langchain</code> , <code>langchain-community</code> , <code>langchain-huggingface</code>
Math verification	<code>sympy</code>
NER	<code>spacy</code> + <code>en_core_web_sm</code>
UI	<code>gradio</code> (Blocks API, Soft theme)
Numerics	<code>numpy</code>
Config	<code>pyyaml</code>